O.S. PRACTICAL EXAM KEY ANSWERS

BATCH-1

1.) **i. Write a Shell program to check the given number is palindrome or not.**

ii. Write a Shell program to find the factorial of a number.

Ans:- i.) AIM

To write a shell program to check whether a given number is a palindrome or not.

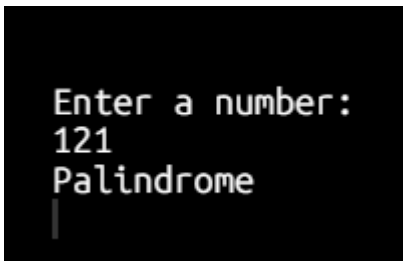
ALGORITHM

1. Read the number from user.
2. Store the original number in a variable.
3. Reverse the number using loop.
4. Compare original number with reversed number.
5. Display result (Palindrome or Not).

PROGRAM

```
echo "Enter a number:"
read num

original=$num
reverse=0

while [ $num -gt 0 ]
do
    digit=$((num % 10))
    reverse=$((reverse * 10 + digit))
    num=$((num / 10))
done

if [ $original -eq $reverse ]
then
    echo "Palindrome"
else
    echo "Not a Palindrome"
fi
```

OUTPUT



Result

The program was successfully executed and verified.

ii.) **AIM**

To write a shell program to find the factorial of a given number.

ALGORITHM

1. Read the number from user.
2. Initialize factorial variable as 1.
3. Use loop from 1 to given number.
4. Multiply each number with factorial.
5. Display the factorial result.

PROGRAM

```
1  echo "Enter a number:"
2  read num
3
4  fact=1
5
6  for (( i=1; i<=num; i++ ))
7  do
8      fact=$((fact * i))
9  done
10
11 echo "Factorial = $fact"
```

OUTPUT

```
Enter a number:
9
Factorial = 362880
```

RESULT

The program was successfully executed and verified.

2.) Write a C program for implementation of Round Robin scheduling algorithms

Ans:- **AIM**

To write a C program to implement Round Robin scheduling algorithm.

ALGORITHM

1. Read number of processes, burst time, and time quantum.
2. Initialize remaining burst time for each process.
3. Execute each process for time quantum in cyclic order.
4. Update remaining time and calculate waiting & turnaround time.
5. Display average waiting time and turnaround time.

PROGRAM

```c
#include <stdio.h>

int main() {
    int n, i, time = 0, remain, tq;
    int wt = 0, tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], rt[n];

    for(i = 0; i < n; i++) {
        printf("Enter burst time for process %d: ", i+1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i]; // remaining time
    }

    printf("Enter time quantum: ");
    scanf("%d", &tq);

    remain = n;

    while(remain != 0) {
        for(i = 0; i < n; i++) {
            if(rt[i] > 0) {
                if(rt[i] <= tq) {
                    time += rt[i];
                    wt += time - bt[i];
                    tat += time;
                    rt[i] = 0;
                    remain--;
                } else {
                    time += tq;
                    rt[i] -= tq;
                }
            }
        }
    }

    printf("Average Waiting Time = %.2f\n", (float)wt/n);
    printf("Average Turnaround Time = %.2f\n", (float)tat/n);

    return 0;
}
```

OUTPUT

```
Enter number of processes: 3
Enter burst time for process 1: 5
Enter burst time for process 2: 4
Enter burst time for process 3: 2
Enter time quantum: 2
Average Waiting Time = 5.33
Average Turnaround Time = 9.00
```

RESULT

The program was successfully executed and verified.

3.) Write a C program for implementation of FCFS scheduling algorithm.

Ans:- AIM

To write a C program to implement First Come First Serve (FCFS) scheduling algorithm.

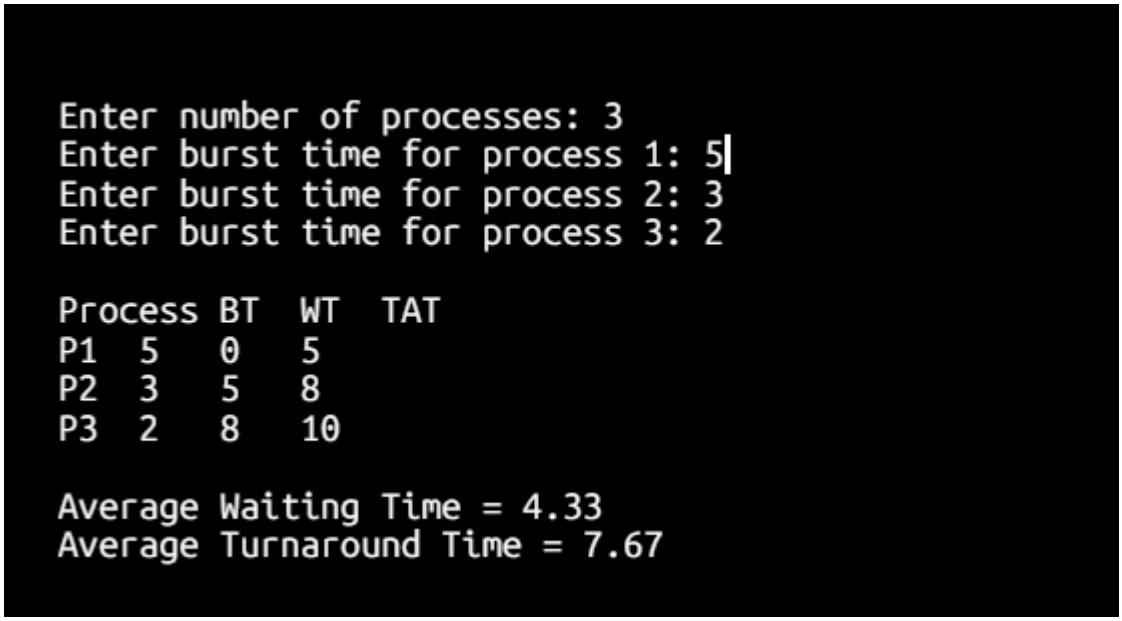
ALGORITHM (Max 5 Steps)

1. Read number of processes and their burst times.
2. Assume arrival order is the same as input order.
3. Calculate waiting time for each process.
4. Calculate turnaround time for each process.
5. Compute and display average waiting time and turnaround time.

PROGRAM

```c
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], wt[n], tat[n];
    float total_wt = 0, total_tat = 0;

    // Input burst times
    for(i = 0; i < n; i++) {
        printf("Enter burst time for process %d: ", i+1);
        scanf("%d", &bt[i]);
    }

    // First process waiting time is 0
    wt[0] = 0;

    // Calculate waiting time
    for(i = 1; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }

    // Calculate turnaround time
    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    // Calculate totals
    for(i = 0; i < n; i++) {
        total_wt += wt[i];
        total_tat += tat[i];
    }

    // Display results
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

OUTPUT



RESULT

The program was successfully executed and verified.

4.) Write a C program for implementation of FIFO page replacement algorithm.

Ans:- **AIM**

To write a C program to implement FIFO (First In First Out) page replacement algorithm.

ALGORITHM

1. Read number of pages, page reference string, and number of frames.
2. Initialize frames as empty.

3. For each page, check if it is already in frames.
4. If not present, replace the oldest page (FIFO order).
5. Count and display total page faults.

PROGRAM

```c
#include <stdio.h>

int main() {
    int n, f, i, j, k, flag, faults = 0;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    int pages[n];
    printf("Enter page reference string: ");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    int frames[f];

    // Initialize frames to -1 (empty)
    for(i = 0; i < f; i++) {
        frames[i] = -1;
    }

    int index = 0;

    printf("\nPage\tFrames\n");

```

```
    for(i = 0; i < n; i++) {
        flag = 0;

        // Check if page is already in frames
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                flag = 1;
                break;
            }
        }

        // If not found, replace using FIFO
        if(flag == 0) {
            frames[index] = pages[i];
            index = (index + 1) % f;
            faults++;
        }

        // Display frames
        printf("%d\t", pages[i]);
        for(k = 0; k < f; k++) {
            if(frames[k] != -1)
                printf("%d ", frames[k]);
            else
                printf("- ");
        }
        printf("\n");
    }
```

```

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

OUTPUT

```
Enter number of pages: 7
Enter page reference string: 1 3 0 3 5 6 3
Enter number of frames: 3

Page    Frames
1   1 - -
3   1 3 -
0   1 3 0
3   1 3 0
5   5 3 0
6   5 6 0
3   5 6 3

Total Page Faults = 6
```

RESULT

The program was successfully executed and verified.

5.) Write a c program to implement Paging technique for memory management.

Ans:- **AIM**

To write a C program to implement paging technique for memory management.

ALGORITHM

1. Read size of logical memory, physical memory, and page size.

2. Calculate number of pages and frames.
3. Read page table (mapping of pages to frames).
4. Accept logical address (page number and offset).
5. Compute physical address using frame number and offset.

PROGRAM

```c
#include <stdio.h>

int main() {
    int logical_mem, physical_mem, page_size;
    int pages, frames;

    printf("Enter size of logical memory: ");
    scanf("%d", &logical_mem);

    printf("Enter size of physical memory: ");
    scanf("%d", &physical_mem);

    printf("Enter page size: ");
    scanf("%d", &page_size);

    // Calculate number of pages and frames
    pages = logical_mem / page_size;
    frames = physical_mem / page_size;

    int page_table[pages];

    printf("Enter page table (frame number for each page):\n");
    for(int i = 0; i < pages; i++) {
        printf("Page %d: ", i);
        scanf("%d", &page_table[i]);
    }

    int page_no, offset;
```

```c
    printf("Enter page number: ");
    scanf("%d", &page_no);

    printf("Enter offset: ");
    scanf("%d", &offset);

    // Validate input
    if(page_no >= pages || offset >= page_size) {
        printf("Invalid logical address\n");
        return 0;
    }

    int frame_no = page_table[page_no];
    int physical_address = frame_no * page_size + offset;

    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

OUTPUT

```
Enter size of logical memory: 16
Enter size of physical memory: 32
Enter page size: 4
Enter page table (frame number for each page):
Page 0: 2
Page 1: 1
Page 2: 3
Page 3: 0
Enter page number: 2
Enter offset: 1
Physical Address = 13
```

RESULT

The program was successfully executed and verified.

6.) Write a c program to implement Threading and Synchronization Applications.

Ans:- **AIM**

To write a C program to implement threading and synchronization using mutex.

ALGORITHM

1. Initialize a shared variable and a mutex lock.
2. Create multiple threads.
3. Each thread locks the mutex before accessing shared data.
4. Update shared variable and unlock the mutex.
5. Join threads and display final result.

PROGRAM

```c
#include <stdio.h>
#include <pthread.h>

int counter = 0;              // shared resource
pthread_mutex_t lock;         // mutex

void* increment(void* arg) {
    for(int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);   // lock
        counter++;                   // critical section
        pthread_mutex_unlock(&lock); // unlock
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    // Create threads
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    // Wait for threads to finish
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value = %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

OUTPUT

```
Final Counter Value = 200000
```

RESULT

The program was successfully executed and verified.

7.) Write a C program for implementation memory allocation methods for fixed partition using first fit.

Ans:- **AIM**

To write a C program to implement memory allocation using First Fit method in fixed partition.

ALGORITHM

1. Read number of memory blocks and their sizes.
2. Read number of processes and their sizes.
3. For each process, scan blocks from beginning.
4. Allocate the first block that is large enough.
5. Display allocation result for each process.

PROGRAM

```c
#include <stdio.h>

int main() {
    int nb, np, i, j;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);

    int block[nb];
    printf("Enter size of each block:\n");
    for(i = 0; i < nb; i++) {
        scanf("%d", &block[i]);
    }

    printf("Enter number of processes: ");
    scanf("%d", &np);

    int process[np], allocation[np];

    printf("Enter size of each process:\n");
    for(i = 0; i < np; i++) {
        scanf("%d", &process[i]);
        allocation[i] = -1; // not allocated
    }

    // First Fit Allocation
    for(i = 0; i < np; i++) {
        for(j = 0; j < nb; j++) {
```

```
            if(block[j] >= process[i]) {
                allocation[i] = j;
                block[j] -= process[i]; // reduce block size
                break;
            }
        }
    }

    // Display result
    printf("\nProcess No.\tProcess Size\tBlock No.\n");
    for(i = 0; i < np; i++) {
        printf("%d\t\t%d\t\t", i+1, process[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
    }

    return 0;
}
```

OUTPUT

```
Enter number of memory blocks: 3
Enter size of each block:
100 500 200
Enter number of processes: 4
Enter size of each process:
212 417 112 426

Process No. Process Size    Block No.
1       212     2
2       417     Not Allocated
3       112     2
4       426     Not Allocated
```

RESULT

The program was successfully executed and verified.

8.) Write a C program to implement Banker’s algorithm for deadlock avoidance.

Ans:- **AIM**

To write a C program to implement Banker’s algorithm for deadlock avoidance.

ALGORITHM

1. Read number of processes, resources, Allocation, Max, and Available matrices.
2. Compute Need = Max − Allocation.
3. Initialize Finish array to false and Work = Available.
4. Find a process whose Need ≤ Work, mark it finished and update Work.
5. Repeat until all processes finish (safe state) or no such process exists (unsafe).

PROGRAM

```c
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);

```

```c
    // Calculate Need matrix
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    int finish[n], safeSeq[n];
    for(i = 0; i < n; i++)
        finish[i] = 0;

    int work[m];
    for(i = 0; i < m; i++)
        work[i] = avail[i];

    int count = 0;

    while(count < n) {
        int found = 0;

        for(i = 0; i < n; i++) {
            if(finish[i] == 0) {
                for(j = 0; j < m; j++) {
                    if(need[i][j] > work[j])
                        break;
                }

```

```
                if(j == m) {
                    for(k = 0; k < m; k++)
                        work[k] += alloc[i][k];

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("System is not in safe state\n");
            return 0;
        }
    }

    printf("System is in safe state\nSafe sequence: ");
    for(i = 0; i < n; i++)
        printf("P%d ", safeSeq[i]);

    printf("\n");

    return 0;
}
```

OUTPUT

```
Enter number of processes: 3
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
Enter Available Resources:
3 3 2
System is not in safe state
```

RESULT

The program was successfully executed and verified.

9th Question Deleted.

10.) Write a C-program to implement the producer – consumer problem using semaphores.

Ans:- AIM

To write a C program to implement Producer–Consumer problem using semaphores.

ALGORITHM

1. Initialize buffer, semaphores (empty, full) and mutex.
2. Create producer and consumer threads.
3. Producer waits on empty, locks mutex, adds item, signals full.
4. Consumer waits on full, locks mutex, removes item, signals empty.
5. Repeat for fixed number of items and display output.

PROGRAM

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int item;
    for(int i = 1; i <= 10; i++) {
        item = i;

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void* consumer(void* arg) {
    int item;
    for(int i = 1; i <= 10; i++) {

        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
```

```
57
58      pthread_join(p, NULL);
59      pthread_join(c, NULL);
60
61      sem_destroy(&empty);
62      sem_destroy(&full);
63      pthread_mutex_destroy(&mutex);
64
65      return 0;
66  }
```

OUTPUT

```
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
```

RESULT

The program was successfully executed and verified.

11.) Write a C program for implementation of SJF scheduling algorithm.

Ans:- AIM

To write a C program to implement Shortest Job First (SJF) scheduling algorithm.

ALGORITHM

1. Read number of processes and their burst times.
2. Sort processes in ascending order of burst time.
3. Calculate waiting time for each process.
4. Calculate turnaround time for each process.
5. Compute and display average waiting and turnaround time

PROGRAM

```c
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], wt[n], tat[n], p[n];

    // Input burst times
    for(i = 0; i < n; i++) {
        printf("Enter burst time for process %d: ", i+1);
        scanf("%d", &bt[i]);
        p[i] = i + 1;
    }

    // Sort processes by burst time (SJF)
    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(bt[i] > bt[j]) {
                // Swap burst time
                int temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                // Swap process numbers
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    // Calculate waiting time
    wt[0] = 0;
    for(i = 1; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }

    // Calculate turnaround time
    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    float total_wt = 0, total_tat = 0;

    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

OUTPUT

```
Enter number of processes: 3
Enter burst time for process 1: 6
Enter burst time for process 2: 2
Enter burst time for process 3: 8

Process BT  WT  TAT
P2  2   0   2
P1  6   2   8
P3  8   8   16

Average Waiting Time = 3.33
Average Turnaround Time = 8.67
```

RESULT

The program was successfully executed and verified.

12.) Write a C program to implement Inter process communication using pipes.

Ans:- **AIM**

To write a C program to implement inter-process communication using pipes.

ALGORITHM

1. Create a pipe using pipe() system call.

2. Use fork() to create a child process.

3. Parent process writes data into the pipe.

4. Child process reads data from the pipe.

5. Display the received message.

PROGRAM

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    char write_msg[] = "Hello from Parent";
    char read_msg[50];

    // Create pipe
    if(pipe(fd) == -1) {
        printf("Pipe failed\n");
        return 1;
    }

    int pid = fork();

    if(pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if(pid > 0) {
        // Parent process
        close(fd[0]); // Close read end
        write(fd[1], write_msg, strlen(write_msg) + 1);
        close(fd[1]);
    } else {
        // Child process
        close(fd[1]); // Close write end
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child received: %s\n", read_msg);
        close(fd[0]);
    }

    return 0;
}
```

OUTPUT

RESULT

The program was successfully executed and verified.

13.) QUESTION DELETED

14.) Write a c program to implement LRU page replacement algorithm.

Ans:- **AIM**

To write a C program to implement Least Recently Used (LRU) page replacement algorithm.

ALGORITHM

1. Read number of pages, page reference string, and number of frames.
2. Initialize frames as empty and track recent usage.
3. For each page, check if it is already in frames.

4. If not present, replace the least recently used page.
5. Count and display total page faults.

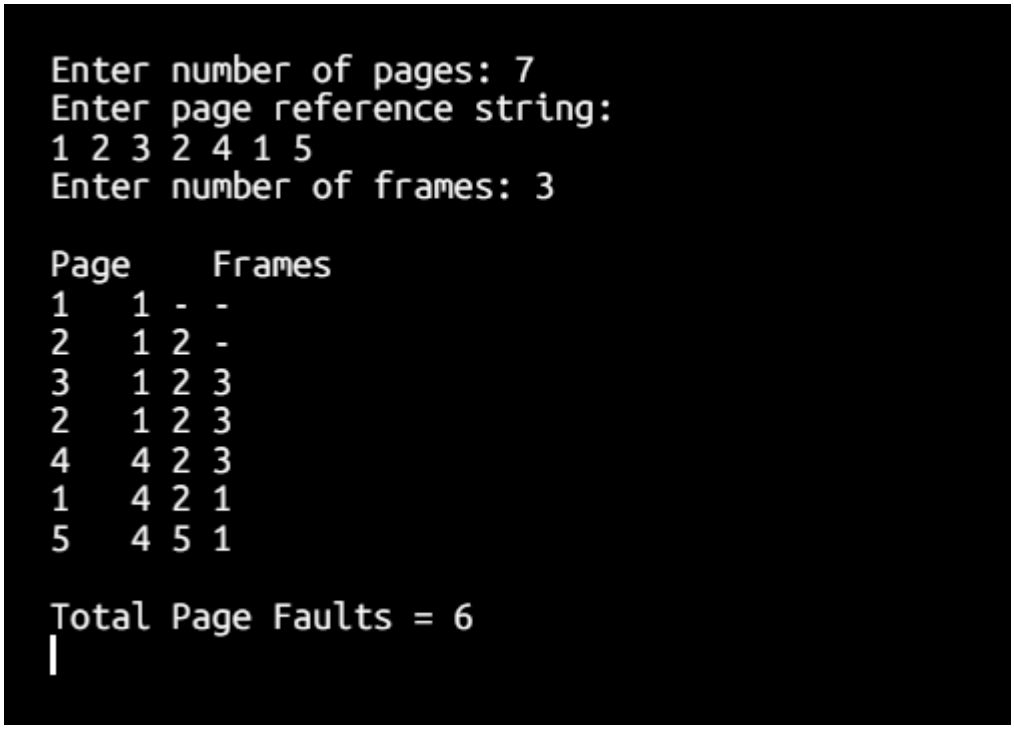
PROGRAM

```c
#include <stdio.h>

int main() {
    int n, f, i, j, k, pos, min, faults = 0;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    int pages[n];
    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    int frames[f], recent[f];

    // Initialize frames
    for(i = 0; i < f; i++) {
        frames[i] = -1;
        recent[i] = -1;
    }

    printf("\nPage\tFrames\n");

    for(i = 0; i < n; i++) {
```

```
        int found = 0;

        // Check if page already exists
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                recent[j] = i; // update recent use
                break;
            }
        }

        // If not found → page fault
        if(!found) {
            faults++;

            // Find empty frame
            for(j = 0; j < f; j++) {
                if(frames[j] == -1) {
                    pos = j;
                    break;
                }
            }

            // If no empty frame, find LRU
            if(j == f) {
                min = recent[0];
                pos = 0;
```

```
                for(k = 1; k < f; k++) {
                    if(recent[k] < min) {
                        min = recent[k];
                        pos = k;
                    }
                }
            }

            frames[pos] = pages[i];
            recent[pos] = i;
        }

        // Display frames
        printf("%d\t", pages[i]);
        for(j = 0; j < f; j++) {
            if(frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

OUTPUT



RESULT

The program was successfully executed and verified.

15.) Write a c program to implement SSTF disk scheduling algorithm.

Ans:- **AIM**

To write a C program to implement Shortest Seek Time First (SSTF) disk scheduling algorithm.

ALGORITHM

1. Read number of disk requests and initial head position.

2. Mark all requests as unvisited.
3. Find the request closest to current head position.
4. Move head to that request and mark it visited.
5. Repeat until all requests are served and calculate total seek time.

PROGRAM

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, i, j, min, pos, head, seek = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    int req[n], visited[n];

    printf("Enter request sequence:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &req[i]);
        visited[i] = 0;
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    for(i = 0; i < n; i++) {
        min = 9999;
        pos = -1;

        // Find nearest request
        for(j = 0; j < n; j++) {
            if(!visited[j]) {
                int diff = abs(head - req[j]);
```

```
                int diff = abs(head - req[j]);
                if(diff < min) {
                    min = diff;
                    pos = j;
                }
            }
        }

        visited[pos] = 1;
        seek += min;
        head = req[pos];
    }

    printf("\nTotal Seek Time = %d\n", seek);

    return 0;
}
```

OUTPUT

```
Enter number of requests: 5
Enter request sequence:
98 183 37 122 14
Enter initial head position: 50

Total Seek Time = 205
```

RESULT

The program was successfully executed and verified.

16.) IS DELETED.

17.) Write C program to implement LFU page replacement algorithm.

Ans:- **AIM**

To write a C program to implement Least Frequently Used (LFU) page replacement algorithm.

ALGORITHM

1. Read number of pages, page reference string, and number of frames.
2. Initialize frames, frequency array, and page fault counter.
3. For each page, check if it is already in frames.
4. If not present, replace the page with the least frequency.
5. If tie occurs, replace the least recently used among them.

PROGRAM

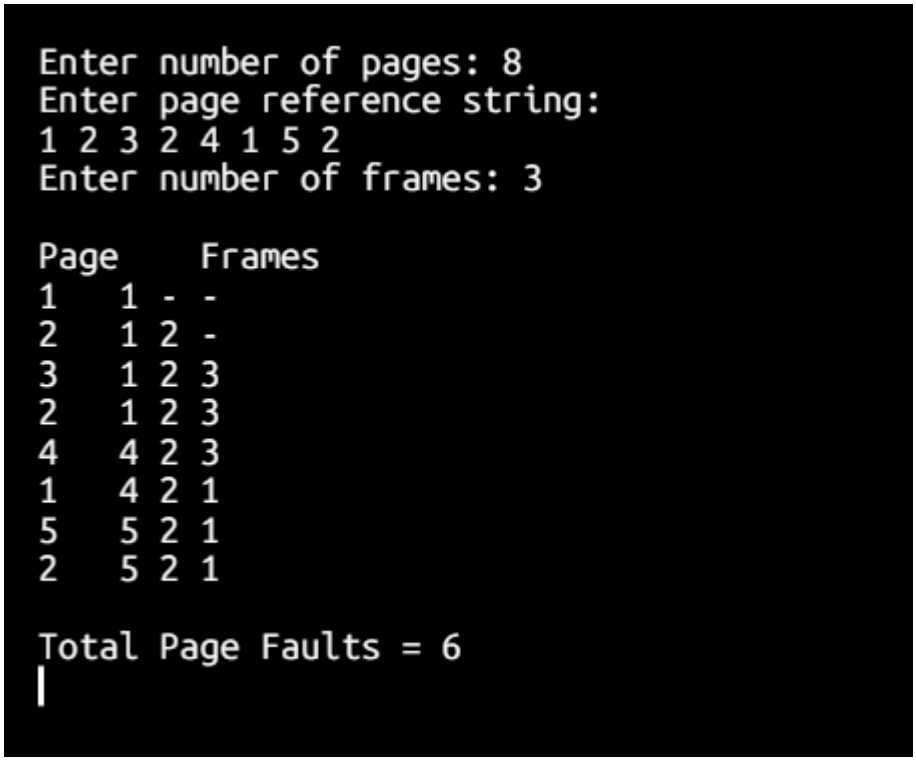
```c
#include <stdio.h>

int main() {
    int n, f, i, j, min, pos, faults = 0;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    int pages[n];
    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    int frames[f], freq[f], time[f];

    // Initialize
    for(i = 0; i < f; i++) {
        frames[i] = -1;
        freq[i] = 0;
        time[i] = 0;
    }

    printf("\nPage\tFrames\n");

```

```c
    for(i = 0; i < n; i++) {
        int found = 0;

        // Check if page already exists
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                freq[j]++;
                found = 1;
                time[j] = i;
                break;
            }
        }

        // Page fault
        if(!found) {
            faults++;

            // Find empty frame
            for(j = 0; j < f; j++) {
                if(frames[j] == -1) {
                    pos = j;
                    break;
                }
            }

            // If no empty frame, apply LFU
            if(j == f) {
                min = freq[0];
```

```
            pos = 0;

            for(j = 1; j < f; j++) {
                if(freq[j] < min) {
                    min = freq[j];
                    pos = j;
                }
                // Tie breaker: LRU
                else if(freq[j] == min && time[j] < time[pos]) {
                    pos = j;
                }
            }
        }

        frames[pos] = pages[i];
        freq[pos] = 1;
        time[pos] = i;
    }

    // Display frames
    printf("%d\t", pages[i]);
    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
    printf("\n");
```

```
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

OUTPUT :-



RESULT

The program was successfully executed and verified.

18.) IS DELETED

19.) Write a c program to implement SCAN disk scheduling algorithm.

Ans:- AIM

To write a C program to implement SCAN (Elevator) disk scheduling algorithm.

ALGORITHM

1. **Read number of requests, request sequence, initial head position, and disk size.**
2. **Sort the request array in ascending order.**
3. **Move head towards one end (0 or max disk) servicing requests.**
4. **After reaching the end, reverse direction and continue servicing.**
5. **Calculate and display total seek time.**

PROGRAM

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, i, j, head, disk_size, seek = 0, direction;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    int req[n + 2];

    printf("Enter request sequence:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &req[i]);
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter disk size: ");
    scanf("%d", &disk_size);

    printf("Enter direction (0 = left, 1 = right): ");
    scanf("%d", &direction);

    // Add boundary values
    req[n] = head;
    req[n + 1] = (direction == 1) ? disk_size - 1 : 0;
```

```
    n += 2;

    // Sort requests
    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(req[i] > req[j]) {
                int temp = req[i];
                req[i] = req[j];
                req[j] = temp;
            }
        }
    }

    // Find head position
    int pos;
    for(i = 0; i < n; i++) {
        if(req[i] == head) {
            pos = i;
            break;
        }
    }

    printf("\nSeek Sequence:\n");

    if(direction == 1) {
        // Move right
        for(i = pos; i < n; i++) {
            printf("%d ", req[i]);
        }
        for(i = pos - 1; i >= 0; i--) {
            printf("%d ", req[i]);
        }
    } else {
        // Move left
        for(i = pos; i >= 0; i--) {
            printf("%d ", req[i]);
        }
        for(i = pos + 1; i < n; i++) {
            printf("%d ", req[i]);
        }
    }

    // Calculate seek time
    seek = abs(head - req[0]) + abs(req[n - 1] - req[0]);

    printf("\nTotal Seek Time = %d\n", seek);

    return 0;
}
```

OUTPUT

```
Enter number of requests: 5
Enter request sequence:
98 183 37 122 14
Enter initial head position: 50
Enter disk size: 200
Enter direction (0 = left, 1 = right): 1

Seek Sequence:
50 98 122 183 199 37 14
Total Seek Time = 221
```

RESULT

The program was successfully executed and verified.

20.) IS DELETED.